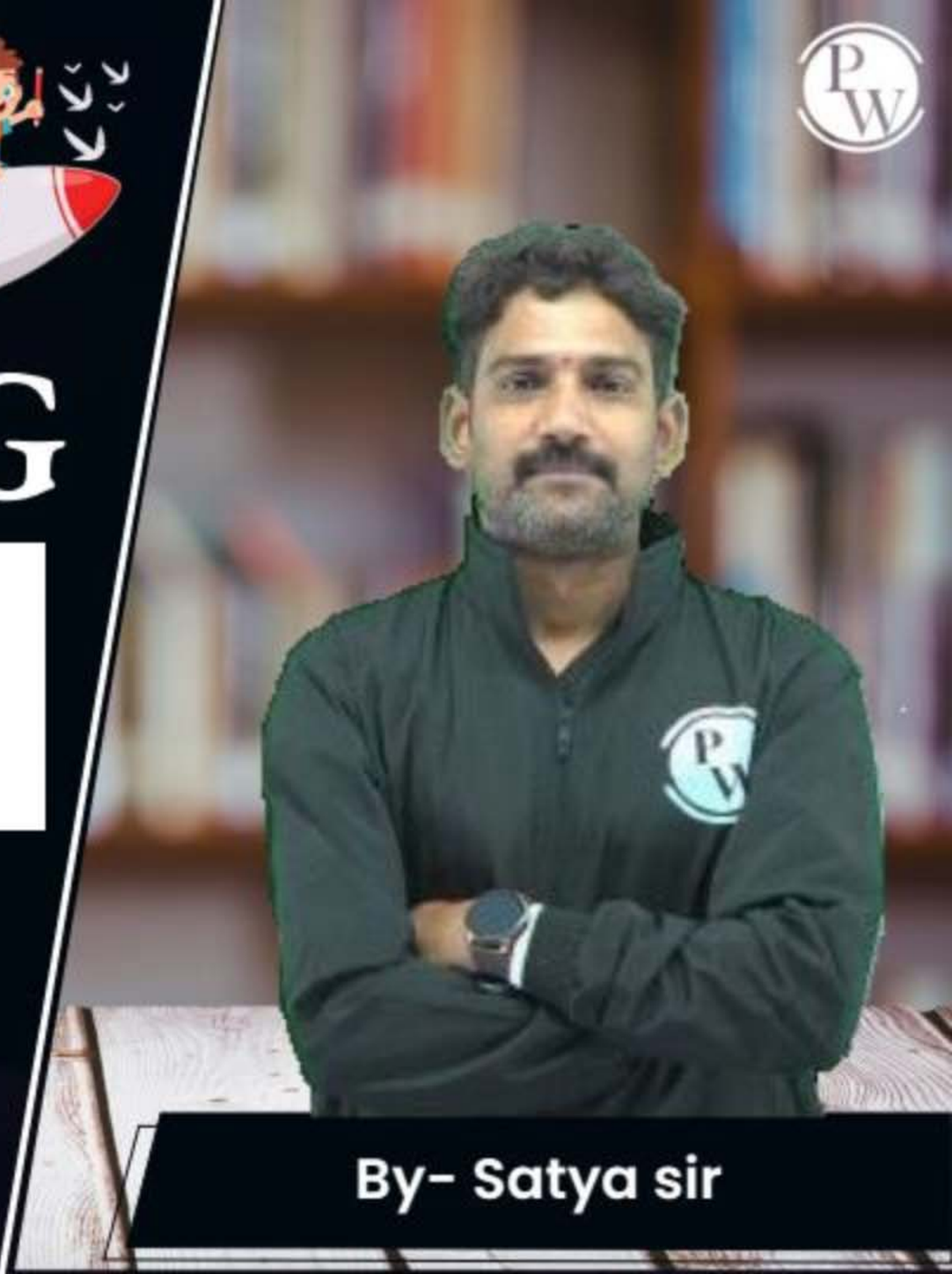
CS & IT ENGINEERING

Data Structure & Programming

Stack and Queues

Lecture No.- 04



By- Satya sir

Recap of Previous Lecture

- Expression Conversion
  - Infix to Postfix
  - Infix to Prefix
- Expression Evaluation
  - Postfix Exp
  - Prefix Exp

Topics to be Covered

- Practice on Exp Conversion, Evaluation
- Queues
  - Nature
  - Types
  - Operations
  - Simple Queue
  - Circular Queue

Topic : Expression Evaluation

H/W :

Infix to Postfix →

X : 7 + (3 * 5) − (9 / 2) + (4 ^ 2) − 8

← Infix to Prefix

Infix to Postfix Conversion

S | + | (| * |) | − | (| / |) | + | (| ^ |) | − |

Pop Pop Pop Pop Pop Pop Pop

Postfix Exp y : 7 3 5 * + 9 2 / − 4 2 ^ + 8 −

Evaluation

S | 7 | 3 | 5 | 15 | 22 | 9 | 2 | 4.5 | 18 | 4 | 2 | 16 | 34 | 8 | 26 |

(*) (/) (^) (−) (+) Pop Result

Infix to Prefix Conversion

A ^ B

S | − | (| ^ |) | + | (| / |) | − | (| * |) | + |

Pop Pop Pop

Intermediate Exp, y : 8 2 4 ^ 2 9 / 5 3 * 7 + − + −

Prefix Exp Z : − + − + 7 * 3 5 / 9 2 ^ 4 2 8

Evaluation

− + | 8 | 2 | 4 | 16 | 7 | 9 | 11 | 5 | 3 | 15 | 7 | 22 | 18 | 34 | 26 |

(^) (/) (*) (+) (−) Pop Result

Topic : Types Of Queues, Operations

- Infix to Postfix/Prefix Conversions use operator Stack.
- Postfix/Prefix Expression Evaluation Use Operand Stack.

Consider an Infix Expression $a + b * c / d \wedge e - f * g$ (Low, high)

with Precedence and associativity as below:

	Operator	Precedence	Associativity
High	$\wedge$	1	R To R
	+	2	L To R
	/	3	L To R
	-	4	L To R
Low	*	5	R To L

The Resultant Postfix Exp and Prefix Exp are —

Infix to Postfix

S: | + | * | / | $\wedge$ | - | * |

y: a b + c d e $\wedge$ / f - g * *

Infix to Prefix conversion

S: | * | - | $\wedge$ | / | * | + |

y: g f e d $\wedge$ c / - * b a + *

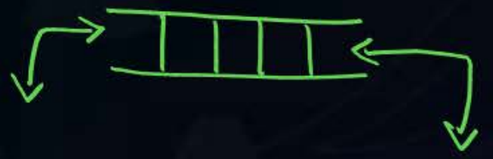
Z: * + a b * - / c $\wedge$ d e f g

Topic : Types Of Queues, Operations

QUEUE

- A Linear Data Structure
- It allows Insertion and deletion of Elements either from same End or from different Ends.
- Insertion → Enqueue( ), Deletion → Dequeue( )

- Types of Queues



1) Simple Queue
2) Circular Queue

} Enqueue( ) from one End : rear End
Dequeue( ) from other End : front End
} First-In-First-Out list

3) Double Ended Queue : Enqueue(), Dequeue() from both Ends.
   - → Enqueue, Dequeue from same End : LIFO
   - → Enqueue, Dequeue from different Ends : FIFO
4) Priority Queue : Dequeue() is based on Priority

Topic : Types Of Queues, Operations

#Q. Consider an Empty Queue Q, Perform below operations.

- Enqueue(10, Q) ✓
- Enqueue(20, Q) ✓
- Dequeue(Q) ✓
- Enqueue(30, Q) ✓
- Dequeue(Q) ✓
- Enqueue(40, Q)
- a = dequeue(Q)
- Enqueue(50, Q)
- b = dequeue(Q)

b - a = 40 - 30 = 10

~~10~~	~~20~~	~~30~~	~~40~~	50

a = 30 b = 40

Topic : Types Of Queues, Operations

#Q. Consider 2 Queues Q1 and Q2 with elements [5,1,3,2], [2,4,3,7] inserted in the same order respectively.

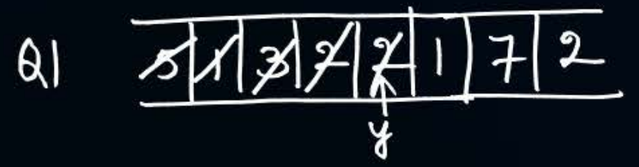
Now, Consider Performing below operations:

1) Enqueue(Dequeue(Q1), Q2) ✓
2) Enqueue(Dequeue(Q2), Q1) ✓
3) Enqueue(Dequeue(Q2), Q2) ✓
4) Enqueue(Dequeue(Q1), Q1) ✓
5) Dequeue(Q1) ✓
6) Dequeue(Q2) ✓
7) Enqueue(Dequeue(Q2), Q1) ✓
8) x = Dequeue(Q2) # x = 5
9) Enqueue(Dequeue(Q1), Q1)
10) y = dequeue(Q1) # y = 2

x + y = 5 + 2 = 7

Q1 = [1, 7, 2]

Q2 = [4]



Topic : Types Of Queues, Operations

#Q. Consider Stack S with Elements 7, 3, −2, 1, 5 in the bottom to top order. A Queue, Q with Elements 5, 3, −2, 1, 7 inserted in the same order. Now, Perform below operations.

1. Enqueue(dequeue(Q), Q) ✓
2. Push(Dequeue(Q), S) ✓
3. Push((Pop(S) + Pop(S)), S) ✓
4. Enqueue(Pop(S), Q) ✓
5. Dequeue(Q) ✓
6. Pop(S) ✓
7. Push(Dequeue(Q), S) ✓
8. Enqueue([Dequeue(Q) + Pop(S)], Q) ✓

bottom top

Resultant Stack, S = 7 3 −2

Resultant Queue, Q = [5 8 8]

2 mins Summary

- Exp Conversion, Evaluation Practice
- Queue DS
  - Nature
  - Types
  - Examples

To be Contd ...

THANK - YOU